ECE 1155: Information Security

Homework 4 & Mini-Lab 3

Note on Academic Integrity:
Acceptable behaviors for this assignment include: Asking your professor or TAs for clarification, consulting the resources posted in Canvas, looking for documentation provided by programming language (e.g., Matlab documentation), discussing with your teammates.

Unacceptable behaviors for this assignment include: Obtaining or copying solutions from anywhere or from others, either in person or online; sharing or posting solutions, either in person or online; asking someone to complete the assignment for you; using artificial intelligence tools in ways not previously authorized by your instructor.

When submitting homework and labs to Gradescope, you must assign the pages for each question to facilitate more efficient grading. Failure to do so will result in a 2% deduction.

Copyright - University of Pittsburgh. All rights reserved. No distribution without written permission

Review Questions: You are not required to submit answers to any of these questions, but you should be able to answer them prior to beginning the homework assignment.

RQ1: What is the relationship between the private key and the public key in RSA? Specifically, show that the private key d should be the multiplicative inverse of the public key e (mod , i.e., mod totient of n).

RQ2: In the chip technology used in credit cards, how does the terminal know that a credit card is issued by an authorized bank and that the card is a valid card?

RQ3: A certificate of an entity may include multiple certificates inside it. When would this happen and why?

RQ4: Who signs the certificate of the root certificate authority (CA)?

RQ5: What are the main properties of a cryptographic hash?

RQ6: Although a primary feature of cryptographic hash is collision resistance, why can hash collisions still occur?

RQ7: What is the implication of the Birthday paradox on cryptographic hashes?

RQ8: For hash functions. What does MD stand for? What does SHA stand for?

RQ9: What are the basic arithmetic and logical functions used in SHA?

RQ10: State two examples of popular hash algorithms and state the number of bits in a hash for each.

RQ11: Explain why we need HMAC for message integrity validation in communication networks.

RQ12: Describe how the HMAC is utilized by a device to validate the integrity of messages received by another distant device. Explain the processes at the sender and receiver.

Copyright - University of Pittsburgh. All rights reserved. No distribution without written permission

Homework Questions: Please answer each of the following questions and submit your answers to Gradescope. Remember to assign pages for each of your answers.

Q1: Suppose that messages are in the form of a sequence of decimal numbers Suppose that we define a hash value h to be calculated as
$h = \Sigma_{i=1}^{t} a_i \, mod \, n,$
for some predefined value of n. Is this a valid cryptographic hash? Justify your answer.

Q2: For an N-bit hash. What is the expected number of hashes that must be computed to find M hash collisions? Evaluate when M = 4, 16, 100.

Q3: Give an example scenario that shows how the Birthday paradox can be exploited by an attacker to compromise a primary objective of cryptographic hash functions.

Q4: Below is an X.509 public-key certificate.

```
Certificate:
 Data:
   Version: 3 (0x2)
   Serial Number:
     3d:0e:98:b2:bf:af:fa:9e:99:91:05:64:69:6e:11:2a
 Signature Algorithm: sha256WithRSAEncryption
   Issuer: C=US, O=Symantec Corporation,
           OU=Symantec Trust Network,
           CN=Symantec Class 3 EV SSL CA - G3
   Validity
      Not Before: Aug 14 00:00:00 2017 GMT
      Not After : Sep 13 23:59:59 2018 GMT
   Subject: ... C=US/postalCode=22230, ST=Virginia,
         L=Arlington/street=4201 Wilson Blvd,
         O=National Science Foundation, OU=DIS,
         CN=www.nsf.gov
   Subject Public Key Info:
     Public Key Algorithm: rsaEncryption
       Public-Key: (2048 bit)
         Modulus:
            00:ca:fb:26:78:06:25:b1:9e:67:1d:69:0b:10:06:
            cf:25:b6:7d:de:8e:56:80:e1:1c:38:52:62:43:fd:
            ...
      Exponent: 65537 (0x10001)
 Signature Algorithm: sha256WithRSAEncryption
     4b:0d:62:11:b4:dc:78:09:12:c1:1b:24:ff:98:43:58:1c:54:
     0a:34:be:8f:3f:12:8f:17:4a:fe:5b:26:13:1a:5f:a7:87:ad:
     ...
     ba:2c:10:c7:bc:8b:2c:15:6e:0c:d2:d0:8b:74:52:c8:ed:05:
     0b:9b:62:41
```

Copyright - University of Pittsburgh. All rights reserved. No distribution without written permission

A) Who issues the certificate?
B) Who is the owner of the certificate?
C) Who generated the signature on this certificate? and how can this signature be verified?
D) The main purpose of the certificate is to provide trusted values of cryptographic keys for public-key encryption method. What is the encryption method for which the keys are provided? What is the value of the public key (e and n)? If a number is too large, you only need to write down its first four bytes.
E) Is the private key included in the certificate?
F) Note the signature is obtained by first obtaining the Hash then signing the Hash. What is the signature algorithm in the certificate? What encryption method is used to generate the signature? What is the hash function used to generate the signature?

Copyright - University of Pittsburgh. All rights reserved. No distribution without written permission

Lab Tasks: Please answer each of the following questions and submit your answers to Gradescope. Remember to assign pages for each of your answers. Please also attach screenshots showing that you completed each of the tasks.

Task 1: Generating Message Digest and MAC

In this task, we will play with various one-way hash algorithms. You can use the above openssl dgst command to generate the hash value for a file. To see the manuals, you can type “man openssl” and “man dgst”.

```
openssl dgst dgsttype filename
```

Replace the dgsttype with a specific one-way hash algorithm, such as -md5, -sha1, -sha256, etc. For example, to obtain SHA-256 hash of file filename.c, type “openssl dgst -sha256 filename.c”. You can find the supported one-way hash algorithms by typing "man openssl".

Follow the following steps:

1. First, create a simple text file (e.g. MyText.txt). For example, you can have the file include “hello world”.
2. Find the MD5, SHA-256 and SHA-512 hash of the file you created using the openssl command.
    a. What is the length of the output hash for each?
3. Make a small one-bit change in the file and reevaluate the hash values in 2. How much change is in the hash due to this small change in the file?

Hint: To get a length of a “string”, you can write any code to get length or use terminal command: $ echo -n “string” | wc -c

Task 2: Keyed Hash and HMAC

In this task, we would like to generate a keyed hash message authentication code (i.e. HMAC) for a file. We can use the -hmac option (this option is currently undocumented, but it is supported by openssl). The following example generates a keyed hash for a file using the HMAC-MD5 algorithm. The string following the -hmac option is the key. You can change the key by choosing a different string

```
% openssl dgst -md5 -hmac ‘key’ filename
```

Copyright - University of Pittsburgh. All rights reserved. No distribution without written permission

Please generate a keyed hash using HMAC-MD5, HMAC-SHA256, and HMAC-SHA1 for any file that you choose.

a. Please try several keys with different length. Do we have to use a key with a fixed size in HMAC? why?

b. Also use different size for content in file (make a long file and a short file) and comment on the length of the keyed hash.

Task 3: Generating Two Files with the Same MD5 Hash

In this task, we will generate two different files with the same MD5 hash values. The beginning parts of these two files need to be the same, i.e., they share the same prefix. We can achieve this using the md5collgen program, which allows us to provide a prefix file with any arbitrary content. The way how the program works is illustrated in Figure 1. The following command generates two output files, out1.bin and out2.bin, for a given a prefix file prefix.txt:

```
$ md5collgen -p prefix.txt -o out1.bin out2.bin
```

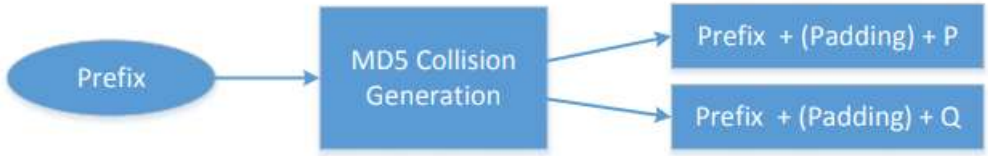

Figure 1: MD5 collision generation from a prefix

We can check whether the output files are distinct or not using the diff command. We can also use the md5sum command to check the MD5 hash of each output file. See the following commands.

```
$ diff out1.bin out2.bin
$ md5sum out1.bin
$ md5sum out2.bin
```

Since out1.bin and out2.bin are binary, we cannot view them using a text-viewer program, such as cat or more; we need to use a binary editor to view (and edit) them. We have already installed a hex editor software called bless (or hex) in our VM. Please use such an editor to view these two output files, and describe your observations. In addition, you should answer the following questions:

A) If the length of your prefix file is not multiple of 64, what is going to happen?
B) Create a prefix file with exactly 64 bytes, and run the collision tool again, and see what happens.
C) Are the data (128 bytes) generated by md5collgen completely different for the two output files? Please identify all the bytes that are different.

Copyright - University of Pittsburgh. All rights reserved. No distribution without written permission

Task 4: Certificate Authority

In this question, you will check the CA (certificate authority) stored in your browser, and find information such as public key, encryption algorithm and hash function used.

In firefox (you can use local or virtual machine), many CA are already installed. To view them, follow the steps below

1. Open Firefox go to edit > preferences
2. Chose privacy and security (lock icon at right)
3. Scroll down and click on view certificates. A “certificate manager” pop-up window will show up.
4. In the “authorities” tab you will see the list of certificates trusted by firefox.
5. Select “GlobalSignRootCA.crt” and export its certificate.
6. Open the certificate you exported, view details.

    a) What are the hash algorithms used?
    b) What is the public key size?
    c) What is the public key algorithm used?
    d) After you exported the certificate, verify that it is valid. You can use terminal and openssl command verify it. Provide a snapshot. If the verification fails, explain why the verification failed.

Hint: terminal command is *openssl verify –CAfile GlobalSignRootCA.crt GlobalSignRootCA.crt*

Copyright - University of Pittsburgh. All rights reserved. No distribution without written permission